

Российский университет транспорта РУТ (МИИТ)
Высшая инженерная школа (ВИШ)

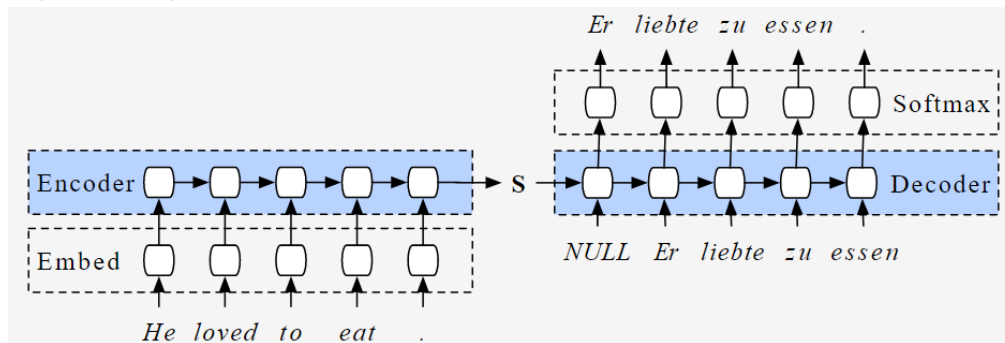
Анализ больших текстовых данных и текстовый поиск

Лекция 11

24 ноября 2025

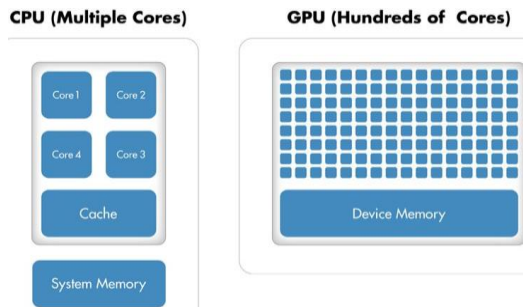
Проблемы RNN и LSTM

- Модели читают последовательность шаг за шагом, что ограничивает скорость обучения и не позволяет эффективно параллелить вычисления
- Обучение плохо справляется с зависимостями, растянутыми по времени: даже LSTM теряет информацию при длинных цепочках
- Градиенты либо затухают, либо взрываются, из-за чего требуется сложная стабилизация
- Контекст держится только в скрытом состоянии, которое передаётся дальше по цепочке и становится узким бутылочным горлышком



Предпосылки к изменениям

- Рост масштабов данных и необходимость обрабатывать длинные тексты, документы, диалоги, код
- Требование к высокой скорости обучения на больших GPU-батчах
- Желание иметь механизм, который может гибко сопоставлять разные части текста, не ограничиваясь локальным окном
- Появление идей внимания в сетях для перевода, которые показали, что прямое сопоставление токенов работает лучше, чем держать всю память в одном векторе



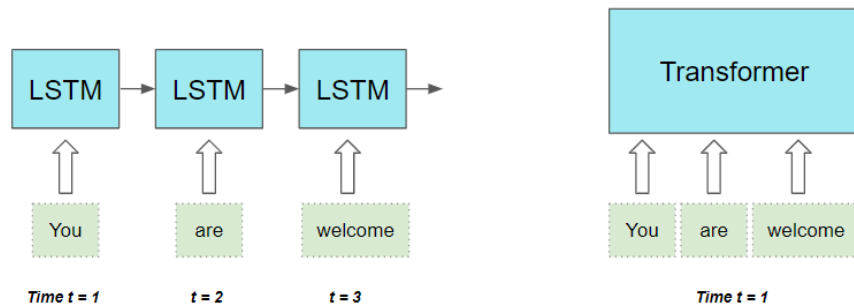
Появление подхода на основе внимания

- В 2017 году предложена архитектура, использующая только операции внимания, без рекурсии и свёрток
- Главная идея — сравнивать каждый токен с каждым напрямую, чтобы модель сама находила связи в предложении
- Переломный момент в NLP: ускорение обучения, рост масштабов моделей, быстрые улучшения качества на задачах перевода и понимания текста
- Этот подход стал основой большинства текущих моделей, включая BERT, GPT и другие.



Ключевая концепция

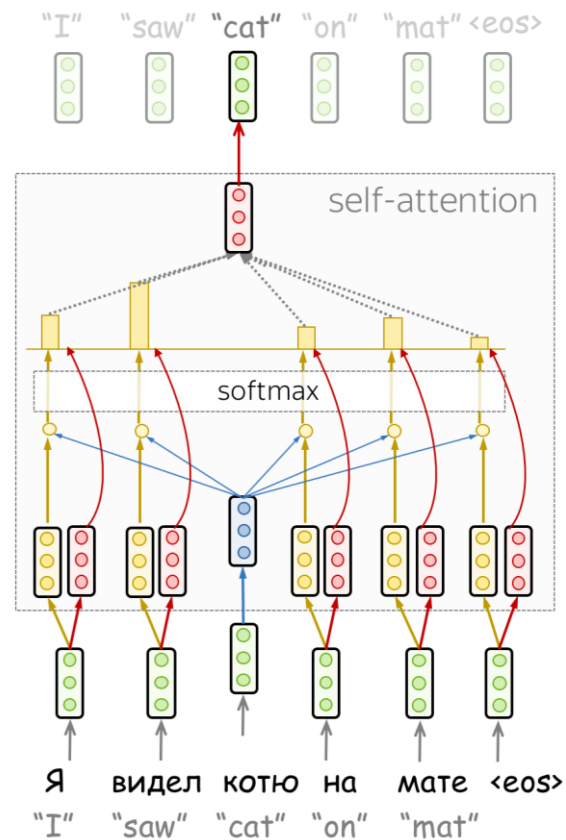
- Модель не проходит последовательность по шагам — все токены обрабатываются одновременно
- Вместо скрытого состояния используется механизм внимания, который позволяет каждому токenu напрямую взаимодействовать с любым другим
- Этот подход устраняет ограничения последовательной обработки и делает обучение масштабируемым



Ключевая концепция

Как представляется входная последовательность:

- Все токены сначала преобразуются в векторные представления одинаковой длины
- К этим векторам добавляется информация о позиции, чтобы модель знала порядок слов
- После этого каждый элемент получает возможность смотреть на другие элементы и взвешивать их вклад



Ключевая концепция

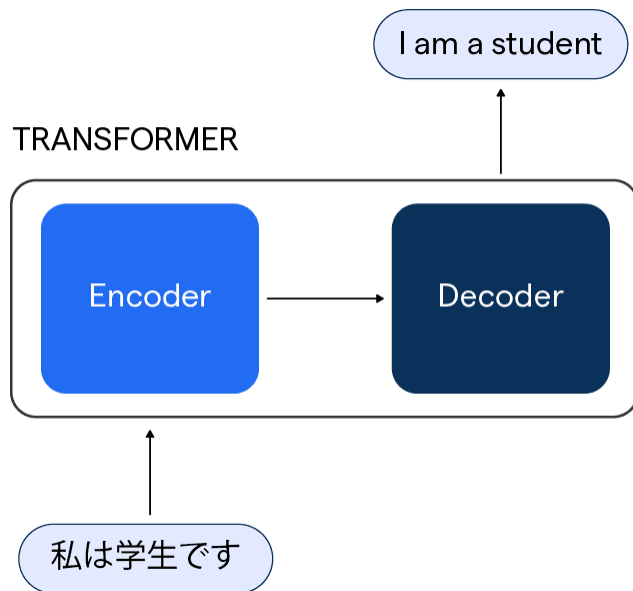
Два основных блока

Encoder

- Преобразует входной текст в набор контекстных представлений, где каждый токен учитывает окружение
- Формирует структурированную информацию о смысле и связях внутри последовательности

Decoder

- Создаёт выходной текст, опираясь на представления, полученные энкодером
- Последовательно формирует токены, используя накопленный контекст и информацию о структуре исходной фразы



Механизм внутреннего внимания (self-attention)

Self-attention позволяет каждому токenu в последовательности посмотреть на все остальные токены и понять, какие из них наиболее важны для его собственного представления

Это не просто поиск похожих слов — это попытка понять, какие части контекста важны именно сейчас

Например, в предложении

Мама мыла раму. Она держала в руках тряпку

Местоимение «она» относится к маме или к раме?

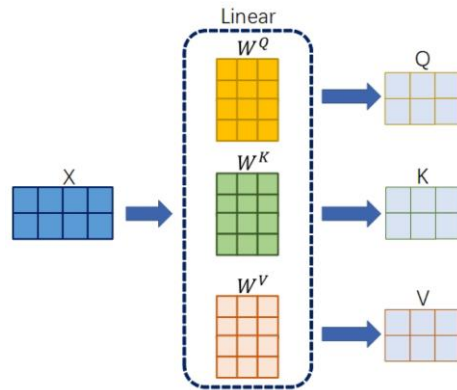
Для человека это очень простой вопрос, но для модели машинного обучения – нет. Self-attention помогает выучить взаимосвязи между токенами в последовательности, моделируя смысл и влияние других слов в тексте при обработке текущего токена.

Механизм внутреннего внимания (self-attention)

Как модель готовит токены к механизму внимания

Каждое слово сначала становится эмбеддингом — вектором фиксированной длины, например 512 элементов. Но для self-attention одного такого вектора недостаточно. Нужно три разных представления одного токена:

Название	Роль
Query (Q) «запрос»	От имени какого токена мы ищем важные части контекста
Key (K) «ключ»	Описание того, насколько этот токен может быть важным для других
Value (V) «значение»	Содержательная информация, которая в итоге будет использована



Все три вектора получаются из исходного эмбеддинга через умножение на обучаемые матрицы W_Q , W_K и W_V . То есть модель сама учится тому, как именно нужно преобразовать эмбеддинг в роли Query, Key и Value.

Механизм внутреннего внимания (self-attention)

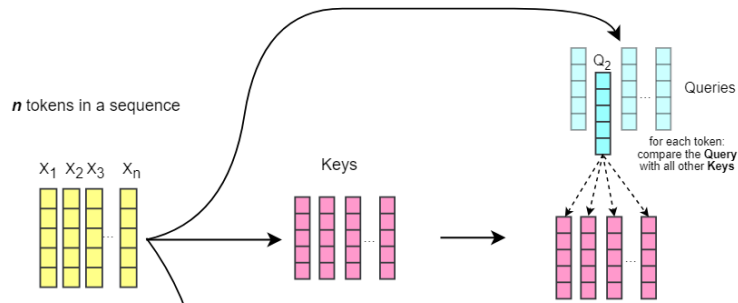
Как токен понимает, какие слова ему важны

Рассмотрим один токен — например, второе слово в предложении. Теперь оно будет пытаться понять, какие другие слова нужно учитывать сильнее, а какие слабее

Механизм:

- Текущий токен берёт свой Query-вектор
- Этот Query по очереди сравнивается со всеми Key-векторами других токенов
- Сравнение выполняется скалярным произведением: чем больше значение, тем важнее этот токен для текущего

Если токены похожи по смыслу или связаны грамматически — число будет выше. Если нет — ниже. Получается таблица значимости каждого токена для текущего.



Механизм внутреннего внимания (self-attention)

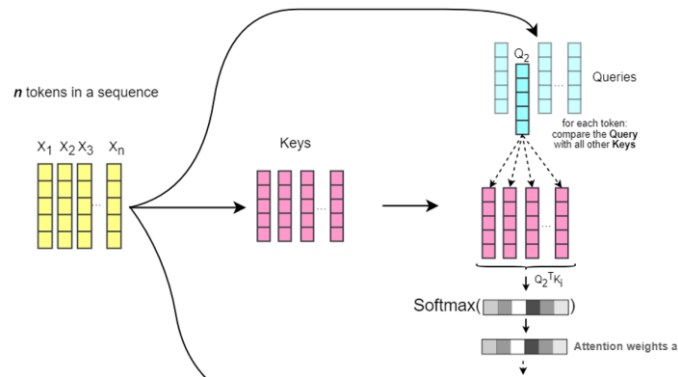
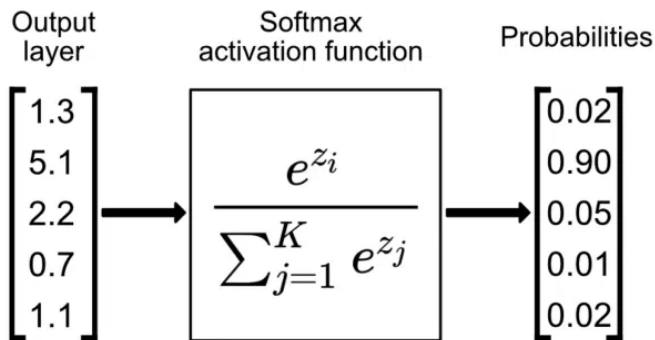
Нормализация: превращаем важности в веса

После сравнения у нас есть набор чисел, но они разнородные, непонятно как их использовать

Нужно превратить их в веса — числа, которые:

- лежат между 0 и 1
- суммируются в 1
- показывают относительную важность каждого токена

Для этого используется softmax. Он оставляет большой вес только у действительно важных токенов, а остальные сильно приглушаются.



Механизм внутреннего внимания (self-attention)

Получение нового представления токена

Теперь мы знаем, какие токены значимы, и насколько.

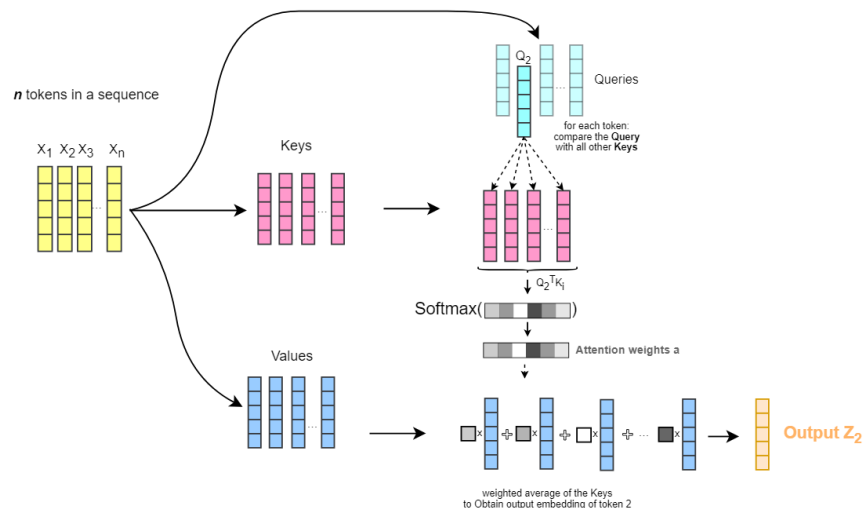
Пора собрать обновлённое представление, которое учитывает весь контекст

Как это делается:

- Берём Value-векторы всех токенов
- Умножаем каждый Value на свой вес внимания
- Складываем эти взвешенные векторы

Получается новый вектор, который содержит информацию со всего предложения, но сфокусирован на самых важных для текущего токена словах.

Это и есть тот самый **self-attention-выход для одного токена**.



Механизм внутреннего внимания (self-attention)

Итоговая формула

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

- QK^T — сходство между Query и Key. Это матрица, в которой каждое число — результат сравнения одного Query-вектора с одним Key-вектором
- $\sqrt{d}$ — масштабирующий коэффициент, стабилизирующий значения. d — размерность векторов Query и Key. Например, если Q и K — это векторы длины 64 или 128, то $d=64$ или $d=128$
- softmax превращает числа в веса
- умножение на V создаёт итоговое контекстное представление

$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{matrix} \square & \square \\ \square & \square \end{matrix} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{matrix} \square \\ \square \end{matrix} \end{matrix}}{\sqrt{d_k}} \right) \begin{matrix} \text{V} \\ \begin{matrix} \square & \square \\ \square & \square \end{matrix} \end{matrix} = \begin{matrix} \text{Z} \\ \begin{matrix} \square & \square \\ \square & \square \end{matrix} \end{matrix}$$

Механизм внутреннего внимания (self-attention)

Суть механизма на примере

Предложение: «*Девочка кормила собаку во дворе, и она громко лаяла*»

Запрос (Query) токена	К чему он присматривается	Почему
«она»	«собаку»	Семантическая связь
«во»	«дворе»	Пространственный контекст
«кормила»	«девочка», «собаку»	Глагол связывает субъект и объект
«лаяла»	«собаку», «она»	Действие и субъект

Модель сама учится это обнаруживать через Q, K, V

Механизм внутреннего внимания (self-attention)

Еще пример с картинкой

Each vector receives three representations (“roles”)

$$\begin{bmatrix} W_Q \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{blue} \\ \text{blue} \\ \text{blue} \end{bmatrix}$$

Query: vector **from** which the attention is looking

“Hey there, do you have this information?”

$$\begin{bmatrix} W_K \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{yellow} \\ \text{yellow} \\ \text{yellow} \end{bmatrix}$$

Key: vector **at** which the query looks to compute weights

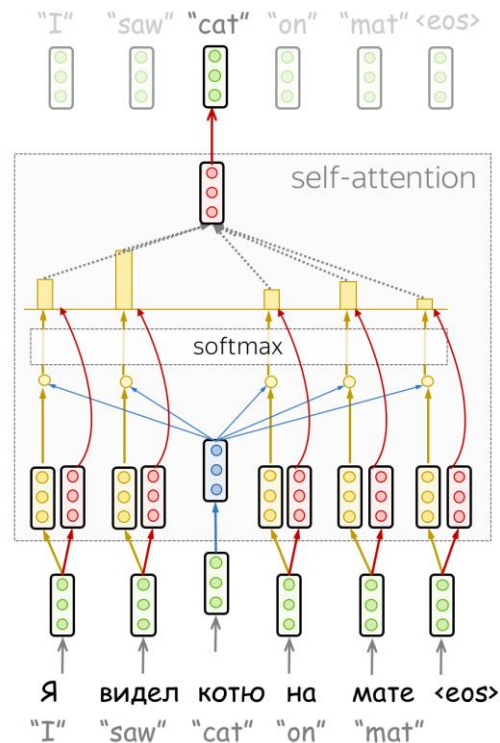
“Hi, I have this information – give me a large weight!”

$$\begin{bmatrix} W_V \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{red} \\ \text{red} \\ \text{red} \end{bmatrix}$$

Value: their weighted sum is attention output

“Here’s the information I have!”

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$



Многоголовый self-attention (multi-head attention)

В трансформерах механизм внимания применяется не один раз, а сразу в нескольких параллельных «головах». Каждая голова — это собственный self-attention блок со своими матрицами весов $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$, поэтому она может выделять собственный тип связей между токенами.

Например:

- одна голова может фокусироваться на грамматических зависимостях
- другая — на смысловых ролях
- третья — на пространственных или временных отношениях

Благодаря параллельной работе голов модель одновременно смотрит на текст с разных сторон, что существенно повышает качество представлений

Многоголовый self-attention (multi-head attention)

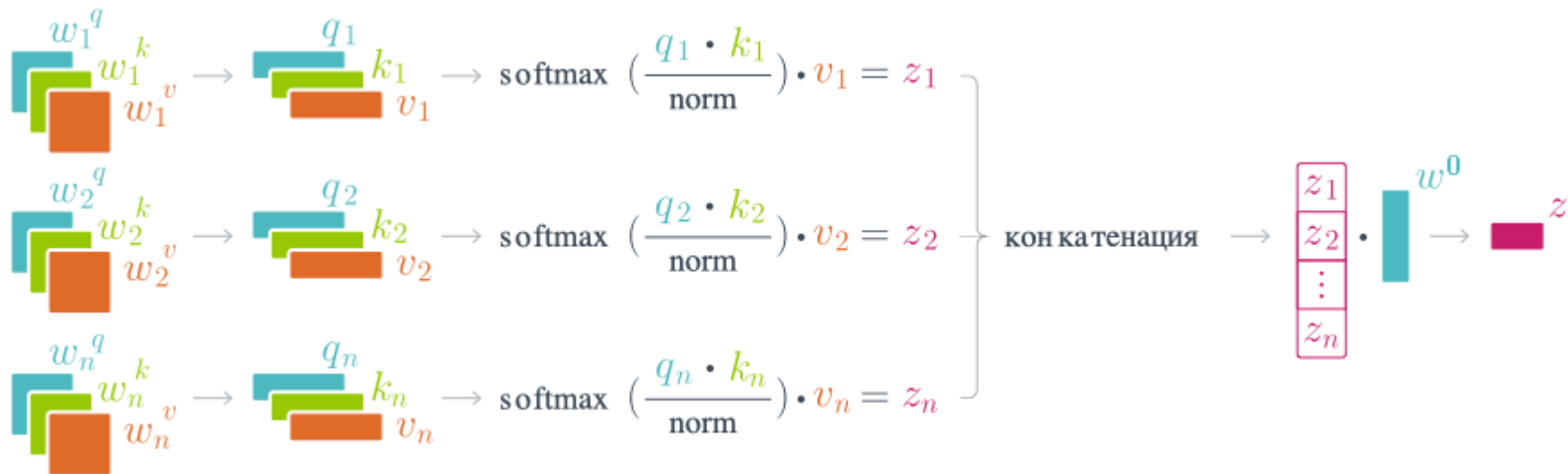
После того как каждая голова вычисляет свой self-attention, результаты объединяют: их конкатенируют по признаковому измерению и пропускают через обучаемую матрицу W_O , которая смешивает выходы разных голов в единое представление. Поскольку все головы работают одновременно, вычисления хорошо распараллеливаются, что ускоряет обучение и делает модель более эффективной.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_n) W_O$$

$$\text{head}_i(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\text{norm}}\right) V$$

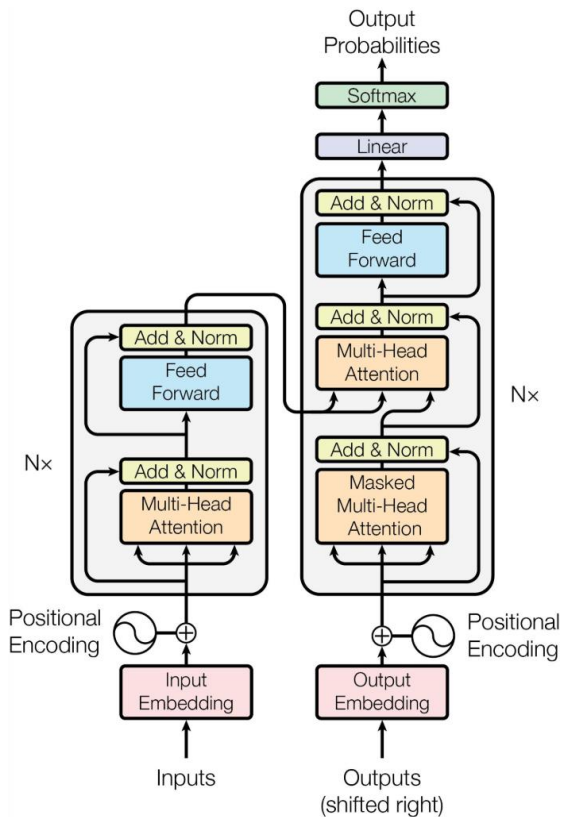
Головы внимания не становятся одинаковыми, потому что каждая из них начинается с разных случайных весов, получает разные градиенты из-за разных Q/K/V-преобразований и нелинейности softmax, а матрица W_O дополнительно поощряет их специализацию, поэтому оптимизация естественным образом разводит головы в разные функциональные роли

Многоголовый self-attention (multi-head attention)



Трансформеры

Трансформеры — это семейство нейронных архитектур, основанных на механизме внимания и не использующих рекуррентные или сверточные слои. Они появились в 2017 году и быстро стали базовым подходом для задач обработки текста, а позже — и для изображений, аудио и других типов данных. Главная мотивация введения трансформеров заключается в ограничениях классических RNN: они хранят всю информацию о последовательности в одном скрытом состоянии и работают строго по шагам, что делает их плохо приспособленными к длинным текстам и затрудняет параллельное обучение. Трансформеры устраняют эти проблемы, заменяя рекуррентность полностью вниманием и позволяя обрабатывать все токены одновременно.



Трансформеры

На анимации показан общий процесс работы трансформера в задаче машинного перевода. На стороне Encoder модель многократно применяет многоголовый self-attention, формируя представления входного текста, которые отражают его структуру и смысловые связи. Далее, на стороне Decoder каждый новый token выходной последовательности генерируется с опорой на полученные представления исходного текста и на уже созданные токены перевода; здесь также используется многоголовое внимание, позволяющее уточнять контекст для каждого шага генерации.

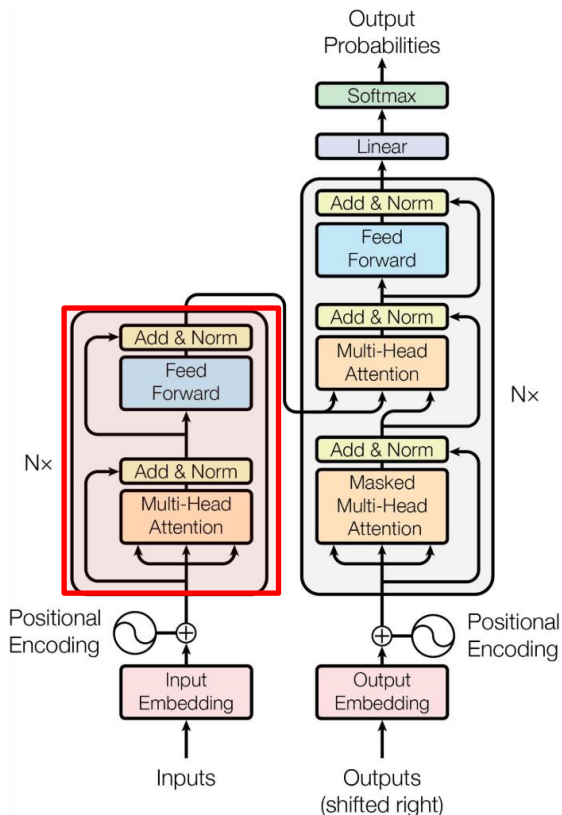


Трансформеры

Encoder (кодировщик)

Состоит из:

- нескольких слоев self-attention (это Multi-Head Attention на схеме). Они позволяют каждому токenu учитывать весь контекст последовательности и формируют уточнённые представления входного текста
- полносвязного слоя (на схеме это Feed Forward). Этот двухслойный блок одинаково применяется ко всем позициям и усиливает выразительность полученных векторов
- слоя Layer Normalization и residual connection (на схеме это Add & Norm) — их мы разберём позже. Остаточные соединения (residual connections) помогают сохранять информацию и стабилизируют обучение, а нормализация выравнивает распределения активаций

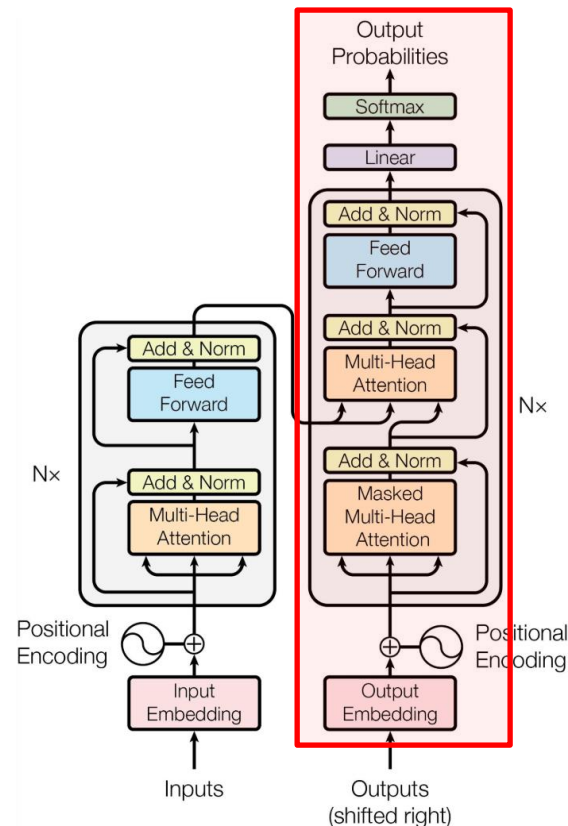


Трансформеры

Decoder (декодировщик)

Состоит из:

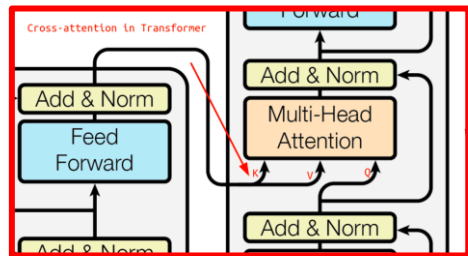
- блока из нескольких self-attention слоёв (на схеме это Masked Multi-Head Attention). Они обрабатывают уже созданные токены выхода и формируют контекст с учётом только доступной части последовательности
- блока из нескольких attention (cross-attention) слоёв (на схеме это Multi-Head Attention). Эти слои позволяют декодировщику обращаться к представлениям кодировщика и использовать информацию из исходного текста
- полносвязного слоя (на схеме это Feed Forward)
- слоя Layer Normalization и residual connection (на схеме это Add & Norm)



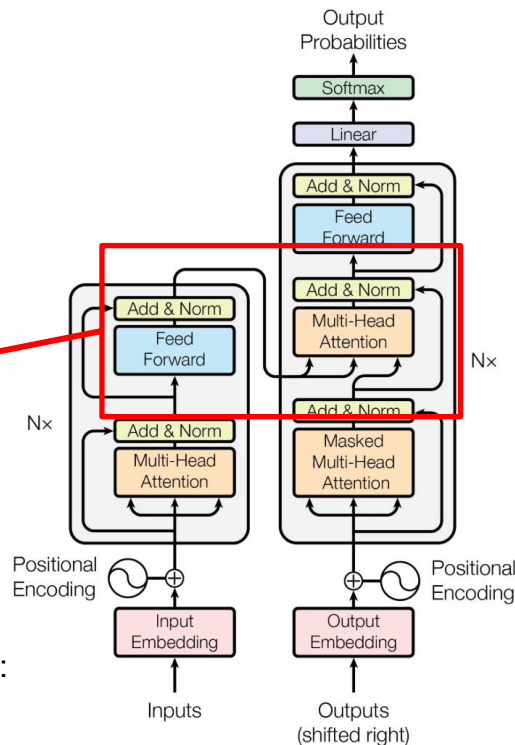
Трансформеры

Отличия decoder от encoder

1. Декодировщик по сравнению с кодировщиком имеет cross-attention блок:



В этом блоке он получает доступ к информации, сформированной кодировщиком: запросы (query) вычисляются из текущего состояния выходной последовательности, а ключи (key) и значения (value) приходят из эмбеддингов, которые выдал кодировщик. Благодаря этому декодировщик может использовать контекст исходного текста при генерации каждого следующего токена.



Трансформеры

2. Первый блок Multi-Head Attention в декодировщике является маскированным (Masked).

Это нужно для того, чтобы при генерации модель не использовала информацию о токенах, которые ещё не созданы. В процессе обучения весь правильный выходной текст известен заранее, и без маски модель могла бы подсматривать будущие токены, что привело бы к утечке информации и сильному переобучению. Чтобы этого избежать, в матрицу внимания вручную зануляют вклад всех позиций, относящихся к будущему. Перед применением softmax соответствующие элементы заменяют очень большими отрицательными значениями, и после нормализации вероятности для этих токенов становятся нулевыми.

the	cake	was	sour
the	cake	was	sour
the	cake	was	sour
the	cake	was	sour

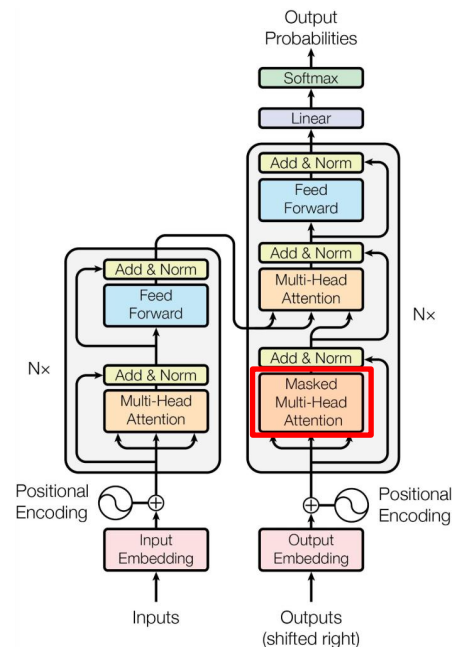
+

0	-inf	-inf	-inf
0	0	-inf	-inf
0	0	0	-inf
0	0	0	0

=

the	-inf	-inf	-inf
the	cake	-inf	-inf
the	cake	was	-inf
the	cake	was	sour

Attention Matrix Masked Matrix Resultant Matrix



Трансформеры

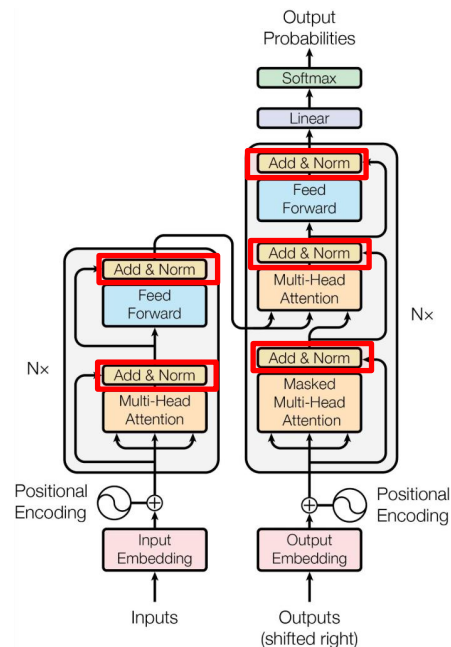
Add & Norm - это сокращение от слоя **Layer Normalisation** и **Residual Connection**

1. Layer Normalization

LayerNorm нормализует вектор признаков каждого токена: выравнивает его среднее и дисперсию, а затем масштабирует и сдвигает результат обучаемыми параметрами. Это помогает стабилизировать значения внутри сети

Для чего он нужен:

- Стабильное обучение. Представления токенов могут сильно скакать по величине, особенно в глубоких моделях. LayerNorm удерживает их в разумных пределах, чтобы градиенты не взрывались и не затухали
- Быстрая сходимость. После нормализации сеть обучается значительно быстрее — особенно трансформеры, где много матриц
- Меньше переобучения. Сглаживает шум и резкие всплески активаций, действуя как лёгкая регуляризация



Трансформеры

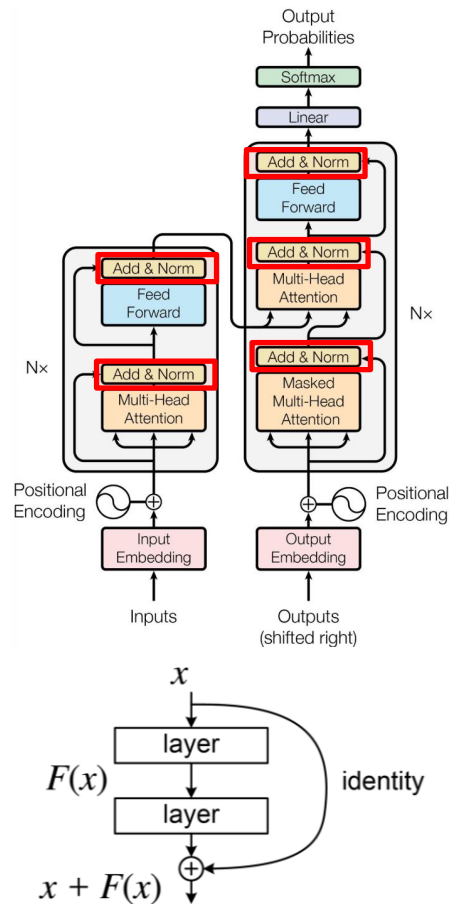
2. Residual connections (остаточные соединения)

В обычных глубоких нейронных сетях градиенты могут становиться слишком малыми, из-за чего ранние слои практически перестают обучаться — это проблема затухающих градиентов. Residual connections решают её, создавая прямой путь от выхода слоя к его входу, что позволяет градиентам свободно распространяться назад и не терять величину даже при наличии большого количества уровней.

Идея заключается в том, чтобы передавать не только результат преобразования $F(x)$, но и сам вход:

$$x + F(x)$$

При подсчете градиента от этого выражения ему сложнее будет затухнуть, так как даже если $F'(x) \approx 0$, то $x' = 1$



Трансформеры

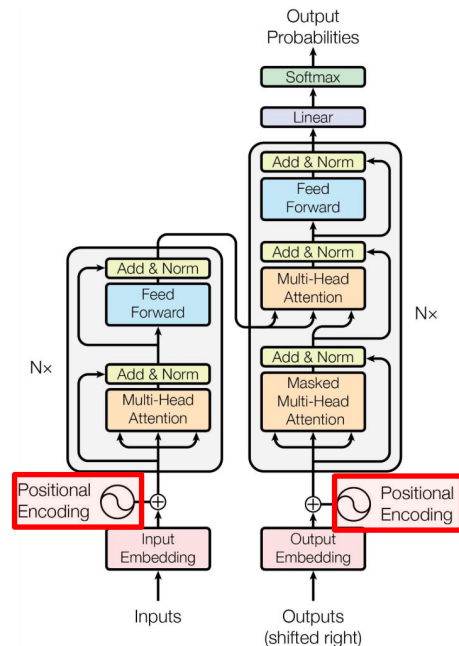
Positional encoding

Positional encoding — это способ сообщить трансформеру информацию о порядке токенов в последовательности, поскольку сама архитектура не читает текст слева направо и обрабатывает все токены параллельно.

Трансформер получает на вход набор эмбеддингов слов, но сам по себе не знает:

- какое слово идёт первым
- какое вторым
- какое в конце
- какие слова соседние

В отличие от RNN, где порядок встроен в саму рекурсию, трансформеру нужно **внешне** передать информацию о позиции токена в предложении



Трансформеры

Positional encoding

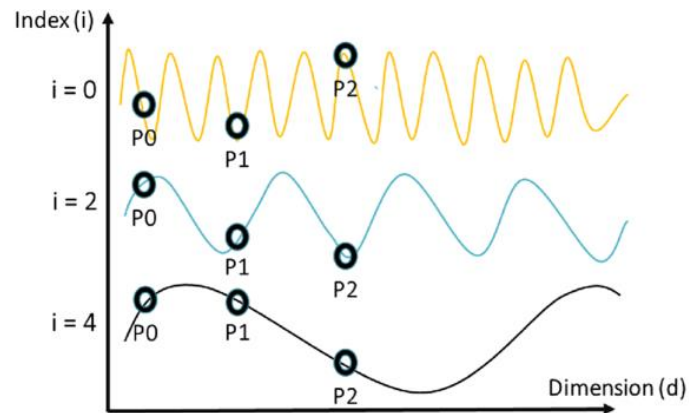
Положение кодируется с помощью синусов и косинусов разных частот.

Так модель получает плавную, бесконечно продолжаемую, математически упорядоченную систему координат.

Преимущества:

- не нужно обучать параметры
- хорошо работают на очень длинных последовательностях
- различия между позициями выражены через периодические функции

Идея: каждой координате вектора positional encoding соответствует своя синусоида или косинусоида с определённой частотой



Трансформеры

Positional encoding

Positional encoding состоит из чередующихся синусов и косинусов:

- чётные измерения — $\sin$
- нечётные измерения — $\cos$

Это делается для того, чтобы модель получала два ортогональных (независимых) сигнала для каждой частоты

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d}} \right)$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d}} \right)$$

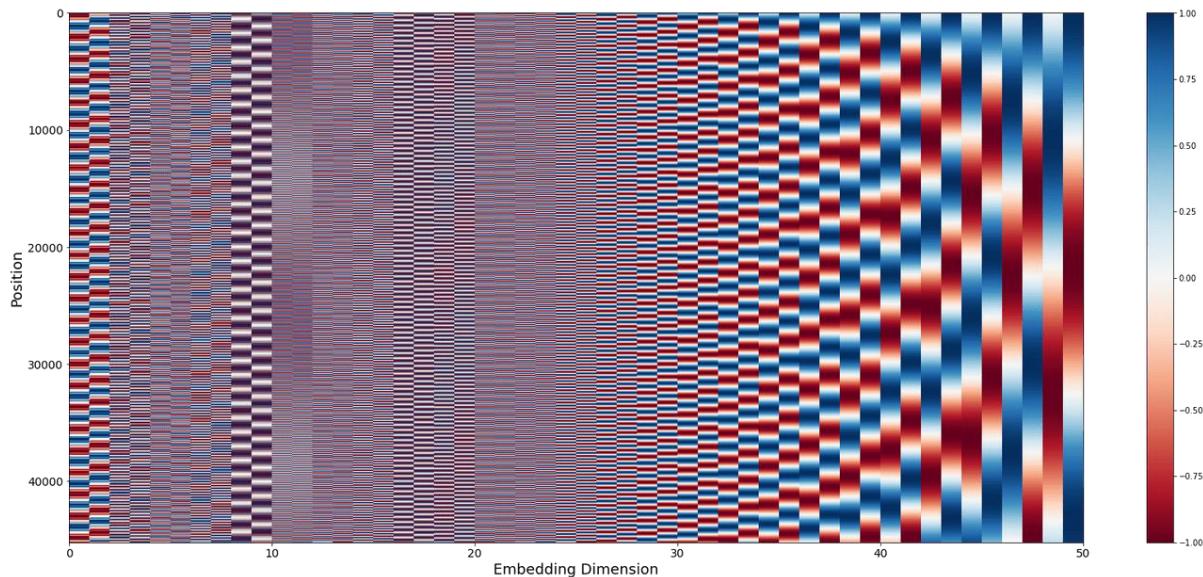
- d — это размерность эмбединга, то есть длина вектора, в котором хранится представление токена.
Positional encoding должен иметь ту же размерность, чтобы его можно было сложить с embedding токена
- i — это индекс измерения внутри этого вектора, то есть номер компоненты Positional encoding

Трансформеры

Positional encoding

На изображении показана тепловая карта синусоидальных positional encodings

Каждая позиция получает уникальный узор, а внимание легко использует эти узоры, чтобы понимать порядок и расстояния между токенами



Трансформеры

Positional encoding

Почему нельзя просто хранить позицию как число?

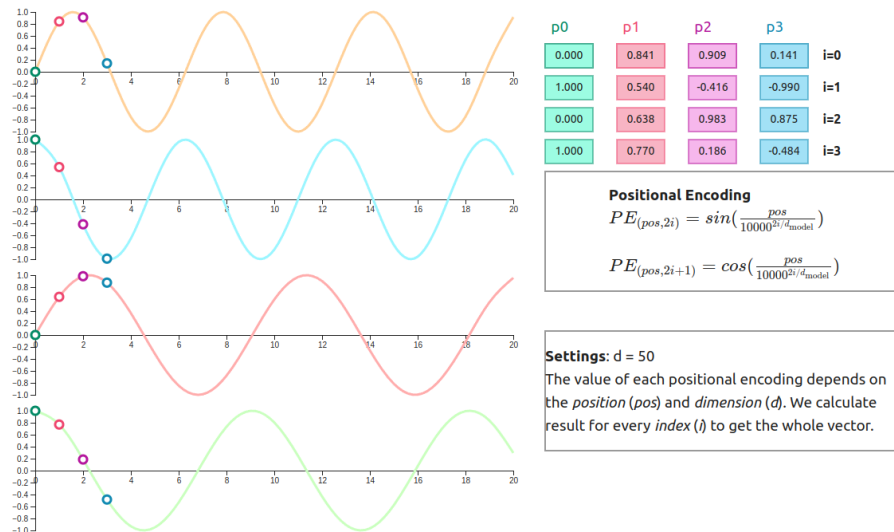
Потому что одно число почти ничего не говорит модели — оно не отражает порядок, расстояние и структуру последовательности

Синусоиды дают многомерный узор, который:

- легко интерпретировать и обрабатывать
- хорошо работает с линейными слоями
- естественно передаёт порядок токенов

Сеть сама учится понимать, что:

- близкие позиции — похожие PE
- далёкие позиции — сильно разные
- расстояние между ними видно как сдвиг волн

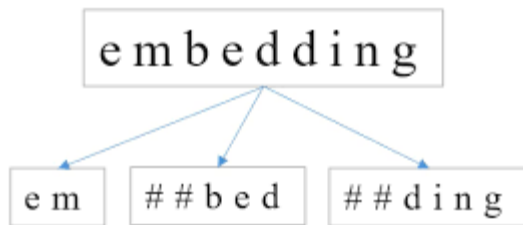


Трансформеры

Byte Pair Encoding (BPE)

В трансформерах (и многих других моделях) текста кодируются при помощи Byte Pair Encoding (BPE). Этот метод нужен затем, чтобы корректно работать с новыми, редкими или ранее не встречавшимися словами. Если такого слова нет в обучающем словаре, модель без BPE присвоила бы ему специальную метку «unknown», и часть информации просто потерялась бы. Но большинство новых или редких слов можно разложить на более простые фрагменты, которые уже есть в словаре. Кодировав эти фрагменты, мы можем представить любое слово и сохранить максимум информации.

Что делает BPE? Часто встречающиеся слова хранятся как цельные токены. А редкие — разбиваются на подслова, из которых формируется словарь. Благодаря этому подходу модель может кодировать любые слова, включая новые формы и неологизмы, без потери смысла.



Трансформеры

Алгоритм **Byte Pair Encoding** включает два этапа:

- обучение — на этом шаге алгоритм формирует правила разбиения слов на подслова (sub-tokens)
- применение — здесь уже готовые правила используются для сегментации текста

Обучение

Изначально словарь состоит из отдельных символов. Далее алгоритм многократно находит пару соседних символов, которые чаще других встречаются вместе в тексте, и объединяет их в новый токен. Этот новый токен добавляется в словарь, после чего процедура повторяется: снова выбирается самая частотная пара токенов, снова создаётся объединение. Процесс продолжается до тех пор, пока словарь не достигнет заданного размера (обычно несколько тысяч элементов)

Count pairs of new tokens:		Words in the data:		Current merge table:	
m + at: 10		c at	4	a t -> at	
at + e: 7	Add the most frequent pair to the merge table	m at	5	m at -> mat	
c + at: 4		m at s	2		
at + s: 2		m at e	3		
e + at: 2		at e	3		
		e at	2		